

Mini-Projet en Langage C

Gestion d'un Stock de Produits
avec Listes Chainees

Rapport Technique

Realise par :	NHARI Mohammed
Encadre par :	M. ABDELWAFI AbdeSamad
Annee scolaire :	2025 / 2026
Niveau :	DAI Premiere Annee

Sommaire

1. Objectif du Projet	3
2. Definition des Structures (Partie 1).....	3
2.1 Forward Declarations.....	3
2.2 Structure produit	3
2.3 Structure maillon.....	3
2.4 Structure stock.....	3
3. Gestion des Produits (Partie 2)	4
3.1 lireProduit().....	4
3.2 afficheProduit()	4
3.3 valeurProduit()	4
4. Gestion du Stock (Partie 3).....	4
4.1 creerMaillon()	4
4.2 initStock() et codeExiste().....	4
4.3 rechercheProduit().....	5
4.4 Ajout en debut et en fin.....	5
4.5 Suppression	5
4.6 Modification	6
5. Affichage et Calculs	6
5.1 afficherStock()	6
5.2 afficherRuptureStock().....	6
5.3 totalStock()	6
5.4 produitLePlusCher()	7
6. Persistence des Donnees (Fichiers Binaires).....	7
6.1 sauvegarderStock()	7
6.2 chargerStock()	7
7. Points Cles & Erreurs Frequentes	8
8. Conclusion	8

1. Objectif du Projet

Ce mini-projet consiste à développer une application complète en langage C permettant de gérer un stock de produits à l'aide de listes chaînées dynamiques. L'objectif pédagogique est de maîtriser les structures de données dynamiques, la gestion de la mémoire avec malloc et free, ainsi que la modularité du code via des fonctions bien définies.

Le programme final couvre la saisie, l'affichage, la recherche, l'insertion, la suppression, la modification des produits, ainsi que la persistance des données via des fichiers binaires.

2. Definition des Structures (Partie 1)

L'architecture repose sur trois structures typées, déclarées en amont pour résoudre les problèmes d'auto-référencement :

2.1 Forward Declarations

```
typedef struct produit produit;  
typedef struct maillon maillon;  
typedef struct stock stock;
```

2.2 Structure produit

```
struct produit {  
    char code[10]; // Identifiant unique  
    char nom[50]; // Nom du produit  
    int qte; // Quantité en stock  
    float prix; // Prix unitaire  
};
```

2.3 Structure maillon

```
struct maillon {  
    produit p; // Données du produit  
    maillon* suivant; // Pointeur vers le noeud suivant  
};
```

2.4 Structure stock

```
struct stock {  
    maillon* tete; // Pointeur vers le premier maillon  
};
```

Note : Le choix des forward declarations permet d'utiliser `maillon*` à l'intérieur de `struct maillon` sans ambiguïté de compilation.

3. Gestion des Produits (Partie 2)

3.1 lireProduit()

Saisit les informations d'un produit. Utilise fgets pour les chaines afin de supporter les espaces, et getchar() pour vider le buffer apres scanf.

```
produit lireProduit() {
    produit p;
    printf("Code: "); scanf("%s", p.code); getchar();
    printf("Nom: "); fgets(p.nom, sizeof(p.nom), stdin);
    p.nom[strlen(p.nom)-1] = '\0';
    printf("Qte: "); scanf("%d", &p.qte); getchar();
    printf("Prix: "); scanf("%f", &p.prix); getchar();
    return p;
}
```

3.2 afficheProduit()

```
void afficheProduit(produit p) {
    printf("|%s\t|s\t|d\t|%.2f\t|\n",
        p.code, p.nom, p.qte, p.prix);
}
```

3.3 valeurProduit()

```
float valeurProduit(produit p) {
    return p.prix * p.qte;
}
```

4. Gestion du Stock (Partie 3)

4.1 creerMaillon()

Alloue dynamiquement un nouveau noeud. Toujours initialiser suivant a NULL pour eviter les pointeurs pendents.

```
maillon* creerMaillon(produit p) {
    maillon* m = (maillon*)malloc(sizeof(maillon));
    m->p = p;
    m->suitant = NULL;
    return m;
}
```

4.2 initStock() et codeExiste()

```
void initStock(stock *s) { s->tete = NULL; }

bool codeExiste(stock s, char code[]) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (!strcmp(ptr->p.code, code)) return true;
        ptr = ptr->suivant;
    }
    return false;
}
```

4.3 rechercheProduit()

Retourne un pointeur direct vers le produit dans le heap — valide tant que le noeud existe.

```
produit* rechercheProduit(stock s, char code[]) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (!strcmp(ptr->p.code, code)) return &(ptr->p);
        ptr = ptr->suivant;
    }
    return NULL;
}
```

4.4 Ajout en debut et en fin

```
void ajouterProduitDebut(stock* s, produit p) {
    if (codeExiste(*s, p.code)) { printf("Erreur\n"); return; }
    maillon* m = creerMaillon(p);
    m->suivant = s->tete;
    s->tete = m;
}

void ajouterProduitFin(stock* s, produit p) {
    if (codeExiste(*s, p.code)) { printf("Erreur\n"); return; }
    if (s->tete == NULL) { s->tete = creerMaillon(p); return; }
    maillon* ptr = s->tete;
    while (ptr->suivant != NULL) ptr = ptr->suivant;
    ptr->suivant = creerMaillon(p);
}
```

4.5 Suppression

```
void supprimerDebut(stock* s) {
    if (!s->tete) return;
    maillon* tmp = s->tete->suivant;
    free(s->tete);
    s->tete = tmp;
}

void supprimerFin(stock* s) {
    if (!s->tete) return;
    if (!s->tete->suivant) { free(s->tete); s->tete = NULL; return; }
    maillon* ptr = s->tete;
    while (ptr->suivant->suivant != NULL) ptr = ptr->suivant;
    free(ptr->suivant);
}
```

```
ptr->suivant = NULL;
}
```

4.6 Modification

```
void modifierProduit(stock* s, char code[]) {
    produit* p = rechercheProduit(*s, code);
    if (!p) { printf("Introuvable\n"); return; }
    char nom[50]; int qte; float prix;
    printf("Nouveau nom: "); fgets(nom, 50, stdin);
    nom[strlen(nom)-1] = '\0';
    printf("Nouvelle qte: "); scanf("%d", &qte); getchar();
    printf("Nouveau prix: "); scanf("%f", &prix); getchar();
    strcpy(p->nom, nom); p->qte = qte; p->prix = prix;
}
```

5. Affichage et Calculs

5.1 afficherStock()

```
void afficherStock(stock s) {
    maillon* ptr = s.tete;
    while (ptr != NULL) { afficheProduit(ptr->p); ptr = ptr->suivant; }
}
```

5.2 afficherRuptureStock()

Filtre uniquement les produits dont la quantité est égale à 0.

```
void afficherRuptureStock(stock s) {
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        if (ptr->p.qte == 0) afficheProduit(ptr->p);
        ptr = ptr->suivant;
    }
}
```

5.3 totalStock()

Note : Toujours initialiser sum à 0 — une variable locale non initialisée contient une valeur indéfinie (garbage).

```
float totalStock(stock s) {
    maillon* ptr = s.tete;
    float sum = 0;
    while (ptr != NULL) {
        sum += valeurProduit(ptr->p);
        ptr = ptr->suivant;
    }
    return sum;
}
```

5.4 produitLePlusCher()

Retourne un pointeur vers le produit le plus cher dans le heap. Le pattern NULL-guard evite toute comparaison invalide a la premiere iteration.

```
produit* produitLePlusCher(stock s) {
    maillon* ptr    = s.tete;
    produit* prdMax = NULL;
    while (ptr != NULL) {
        if (prdMax == NULL || ptr->p.prix > prdMax->prix)
            prdMax = &ptr->p;
        ptr = ptr->suivant;
    }
    return prdMax;
}
```

6. Persistance des Donnees (Fichiers Binaires)

La persistance est assuree par des fichiers binaires. On ecrit les donnees produit noeud par noeud — pas la structure stock (qui ne contient qu'un pointeur memoire sans signification hors processus).

6.1 sauvegarderStock()

```
void sauvegarderStock(stock s, char nomFichier[]) {
    FILE* fp    = fopen(nomFichier, "wb");
    maillon* ptr = s.tete;
    while (ptr != NULL) {
        fwrite(&ptr->p, sizeof(produit), 1, fp);
        ptr = ptr->suivant;
    }
    fclose(fp);
}
```

6.2 chargerStock()

La valeur de retour de fread indique le nombre d'elements lus : 1 tant qu'un produit complet est disponible, 0 a EOF.

```
void chargerStock(stock *s, char nomFichier[]) {
    FILE* fp = fopen(nomFichier, "rb");
    produit p;
    while (fread(&p, sizeof(produit), 1, fp) == 1)
        ajouterProduitFin(s, p);
    fclose(fp);
}
```

Note : *chargerStock reutilise ajouterProduitFin — demonstration concrete de la valeur de la modularite.*

7. Points Cles & Erreurs Frequentes

Erreur Frequent	Explication & Solution
= vs ==	if(s->tete = NULL) assigne au lieu de comparer. Toujours utiliser ==.
&arr vs arr	Un tableau char[] est deja une adresse. Ne pas ecrire &p.code dans scanf.
code vs code	Dans strcmp, passer code (char) et non *code (premier caractere seulement).
Variable non initialisee	float sum; contient une valeur garbage. Toujours ecrire float sum = 0;
Retour adresse locale	Ne jamais retourner &variable_locale — la pile est liberee au retour de fonction.
fclose(filename)	fclose prend un FILE*, pas un nom de fichier. Ecrire fclose(fp).
Double fread	Appeler fread dans la condition ET dans le corps de boucle saute des elements.
supprimerFin ptr nul	S'arreter a ptr->suivant->suivant != NULL, pas ptr->suivant != NULL.

8. Conclusion

Ce projet couvre l'ensemble du cycle de vie d'une structure de donnees dynamique en C : definition des types, allocation memoire, operations CRUD, persistance fichier, et liberation memoire. Les concepts fondamentaux mis en pratique incluent :

- Les listes chainees comme structure dynamique par opposition aux tableaux statiques.
- La gestion memoire manuelle avec malloc et free.
- La modularite pour eviter la duplication de code.
- Les pointeurs sur heap vs pointeurs sur pile.
- La persistance binaire avec fread/fwrite pour la serialisation de structs.